Desafio Prático: Automação de Testes de API Modernas

Ferramentas Necessárias: Python 3.9+, `pytest`, `pytest-asyncio`, `httpx`, `tavern`, `fastapi`.

Contexto

Você foi contratado como Engenheiro de Qualidade (QA) em uma startup inovadora. Seu primeiro desafio é estruturar a base de testes automatizados do sistema. A arquitetura da empresa envolve o consumo de uma API externa (para gerenciamento de tarefas) e um novo microsserviço interno super rápido construído com FastAPI.

Sua missão é dividida em três partes.

Parte 1: Teste de Contrato com Tavern

A startup consome a API pública `https://jsonplaceholder.typicode.com`. Você precisa testar o fluxo de tarefas (To-Dos), garantindo que os dados retornem corretamente e usando um teste declarativo em YAML.

O que você deve fazer:

1. Crie um arquivo chamado `test_fluxo_todos.tavern.yaml`.
2. **Parametrize** o teste para rodar buscando os `todo_id`: 1, 5 e 15.
3. **Stage 1 (Busca da Tarefa Específica):**
   - Faça um `GET` na rota `/todos/{todo_id}`.
   - Valide se o `status_code` é 200.
   - Usando o recurso de **encadeamento (save)**, capture o valor da chave `userId` retornado no JSON e salve-o em uma variável chamada `id_do_usuario`.
4. **Stage 2 (Busca de Todas as Tarefas do Usuário):**
   - Faça um `GET` na rota `/todos?userId={id_do_usuario}` (usando a variável salva no passo anterior).
   - Valide se o `status_code` é 200.
   - **Integração com Python:** Crie um arquivo `utils.py` com uma função chamada `validar_lista_tarefas`. Essa função deve receber a resposta HTTP, verificar se o retorno é uma lista e confirmar se pelo menos uma das tarefas da lista contém a chave `completed` (seja `true` ou `false`). Chame essa função neste stage usando a tag de integração do Tavern (`verify_response_with`).

Parte 2: Teste de Performance Assíncrona

O time de Back-end acabou de criar um endpoint assíncrono para processar pagamentos, mas eles não têm certeza se ele realmente aguenta múltiplas chamadas simultâneas sem travar.

O código da API local (`api_pagamentos.py`) é este:

```python
from fastapi import FastAPI
import asyncio

app = FastAPI()

@app.get("/processar")
async def processar_pagamento():
    await asyncio.sleep(1.5) # Simula um processamento demorado
    return {"status": "pagamento_aprovado"}
```

O que você deve fazer:

1. Crie um arquivo `test_performance.py`.
2. Usando `pytest`, `pytest-asyncio` e `httpx`, crie um teste que se conecte diretamente à aplicação FastAPI na memória (usando `ASGITransport`, sem precisar rodar o Uvicorn).
3. **Parametrize** o teste para disparar: 5, 20 e 50 requisições simultâneas.
4. Utilize `asyncio.gather` para disparar as tarefas de forma assíncrona.
5. Valide se todos os status retornados são `200` e crie um `assert` para garantir que o tempo total de execução do teste (mesmo com 50 requisições) seja **menor que 3.5 segundos**.

Parte 3: Boas Práticas de Repositório

Ao rodar seus testes, o Pytest e o Python geraram várias pastas de cache que não devem ser enviadas para o repositório da empresa.

O que você deve fazer:

1. Crie um arquivo `.gitignore` na raiz do seu projeto.
2. Configure-o corretamente para ignorar qualquer pasta de cache do pytest e do python (como o `__pycache__`), independentemente do nível de profundidade em que elas estejam no projeto.

—

Entregáveis Esperados:

- `utils.py`
- `test_fluxo_todos.tavern.yaml`

Gerado em: 07/10/2026

- api_pagamentos.py
- test_performance.py
- .gitignore

Gerado em: 07/10/2026